Smart Canteen Management System - Technical Documentation

Executive Summary

The Smart Canteen Management System is a modern, full-stack application designed to optimize kitchen and serving operations. It transitions conventional manual workflows into an automated, hands-free, and real-time tracked environment. By leveraging Web Sockets, Voice recognition, and custom Audio APIs, this system creates an ergonomic and seamless operational flow for Cooks, Servers, and Customers.

Architecture & Technology Stack (Minute Details)

The application is separated into two primary microservices interacting over RESTful APIs and WebSockets.

Frontend Technologies

- **Core Framework**: React 18, bundled via Vite for extremely rapid Hot Module Replacement (HMR) and optimized builds.
- **Styling**: Tailwind CSS 3 for responsive and rapid utility-based styling, utilizing a custom dark mode and color scheme.
- **State & Context**: React Context APIs (`AuthContext`) to handle JWT tokens gracefully. Handles automatic redirection when tokens expire.
- **Routing**: `react-router-dom` configuring distinct restricted areas for Owner, Cook, Server, and Customer operations.
- **Real-Time Layer**: `socket.io-client` listening to localized namespace events (`join_canteen` rooms) to reflect changes instantly.
- **HTTP Client**: `axios` with interceptors configured to attach the JWT automatically.
- **Hardware Integrations**:
  - **Web Audio API**: Synthesizing distinct low-latency notification tones without relying on heavy `.mp3` assets.
  - **Web Speech API**: Extracting transcriptions dynamically using `SpeechRecognition` to execute status mutations hands-free. Includes fuzzy pattern matching to detect phrases like "preparing", "making", or "ready".
- **Security for Hardware**: `@vitejs/plugin-basic-ssl` injected to serve local development over `https://` , which is a strict browser requirement for microphone availability.

Backend Technologies

- **Core Framework**: Flask v3.0 (Python).
- **Database Architecture**: SQLite (via `Flask-SQLAlchemy`). Structured with models for `User` , `Canteen` , `MenuItem` , `Order` , `OrderItem` , and `CalendarEvent` .
- **Authentication**: JWT-based secure tokens via `Flask-JWT-Extended` , generating cryptographically verified stateless access tokens. Password hashes secured using `scrypt` from Werkzeug.
- **Real-Time Engine**: `flask-socketio` enabling bi-directional event emission across specific Web Socket rooms linked to individual Canteens.
- **API Endpoints & Logic**: Blueprint architecture separating Domain concerns (Auth, Menu, Orders, Analytics, Forecast). Includes dynamic popularity sorting (Left Joins on `OrderItem`) to surface highly-ordered items on the public guest menu.

Working Flow: An End-to-End Example

To understand how the microservices sync in real-time, consider the following example workflow:

1. **Menu Browsing (Customer)**: A student named "Viraj" scans a QR code taped to a table, opening the `GuestMenu` component link for "MMCOE Canteen".
   - The UI presents items like "Masala Dosa". Because the backend executes a fast SQL aggregation of past orders, Masala Dosa is automatically tagged with a **"🔥 Popular"** badge.
   - Unavailable items are dynamically greyed-out and disabled from ordering so users do not place impossible requests.
2. **Order Placement**: Viraj adds items to the cart, types his name, and selects "Cash at counter".
   - The Frontend sends a `POST` request to `/api/orders/canteen/1`.
   - Backend assigns an atomic daily `token_number` (e.g., Token 42) and saves the order as `pending_payment`.
3. **Owner Counter (Payment)**: On the Owner's iPad (`Counter.jsx`), the screen dynamically fetches pending payments.
   - Viraj gives the Owner ₹60 at the register. The Owner clicks **"Mark Paid"** on Token 42.
   - The Backend (`PATCH /orders/42/payment`) upgrades the status to `accepted` and emits an `order_new` Socket.io event to the canteen's private room.
4. **Kitchen Operations (CookBoard - FIFO)**:
   - The `order_new` socket event triggers the `CookBoard.jsx`. It utilizes the Web Audio API to play a sharp triangle-wave chime.
   - "Token 42" slides into the top of the **New Orders** column, as the system strictly enforces a **First-In-First-Out (FIFO)** queueing visualization.
   - Without wiping flour off their hands, the cook says: ***"I'm making 42."***
   - The Web Speech API transcribes this, checks it against `COOK_START_ACTIONS`, matches it, and sends a `PATCH` request to update status to `preparing`. Token 42 visually hops over to the "Preparing" column.
   - Later, the cook says: ***"Ready 42."*** It moves to the "Ready" column. A socket event `order_update` is sent.
5. **Serving Operations (ServerBoard - FIFO)**:
   - The Server device hears the socket event and uses the Web Audio API to play a double-sine wave chime indicating food is ready in the pass.
   - Token 42 appears at the top of the **Ready to Serve** column (Oldest First / FIFO).
   - The server picks up the Dosa, spots Viraj, and hands it over. The Server speaks: ***"Served 42."***
   - The order completes its total lifecycle, disappearing from active dashboards.

Future Scope & Improvements

While the current system provides robust localized operations, there are several avenues for future enhancement:

1. **Inventory & Raw Materials Management**:
   - Implementing a sub-system to track raw ingredients (e.g., potatoes, oil) and mapping them to `MenuItems`. When a Dosa is ordered, deduct raw inventory quantities automatically.
2. **Automated Payment Gateway**:

    - Integrating standard checkout flows via Razorpay or Stripe to allow true remote ordering, bypassing the Owner's physical "Counter Validation" step entirely for online modes.
3. **Machine Learning Demand Forecasting**:
    - Currently, forecasts operate dynamically but heuristically. This could be augmented by analyzing weather APIs, college calendar holidays, and historical trends to predict how much inventory a cook should prep for morning shifts.
4. **Mobile Application Port**:
    - Rewriting the Front-end in **React Native** to generate packaged iOS and Android Apps. This would allow push notifications for background token updates when a user minimizes their browser.
5. **Database Scale-Out**:
    - Migrating from SQLite to PostgreSQL cluster setups for high availability and concurrent load management specifically for large enterprise deployments where thousands of orders occur simultaneously.